Объектно-ориентированное программирование

4 семестр
Лекция 7.

Содержание лекции:

- LINQ

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

LINQ

(**Language-Integrated Query**) представляет простой и удобный язык запросов к источнику данных. В качестве источника данных может выступать объект, реализующий интерфейс IEnumerable (например, стандартные коллекции, массивы), набор данных DataSet, документ XML.

Основная часть функциональности LINQ сосредоточена в пространстве имен System.LINQ. В проектах под .NET 6+ данное пространство имен подключается по умолчанию.

```
string[] people =
{
   "Tom", "Bob", "Sam", "Tim", "Tomas", "Bill“
 };

// Люди с первой буквой T, упорядоченные по имени
var selectedPeople = people
    .Where(p => p.ToUpper().StartsWith("T"))
    .OrderBy(p => p);

foreach (string person in selectedPeople)
    Console.WriteLine(person);
```

```
Tim
Tom
Tomas
```

Пример для коллекции объектов типа **Person**. Создадим класс человека с 2 свойствами: Имя и Возраст.

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

Создадим коллекцию людей:

```
var people = new Person[]
{
   new("Tom", 25), new("Bob", 32), new("Sam", 54),
   new("Tim", 15), new("Tomas", 19), new("Bill", 75)
};

// Люди с первой буквой T, упорядоченные по возрасту
// В отличие от прошлого примеры мы работаем с объектами

var selectedPeople = people
    .Where(p => p.Name.ToUpper().StartsWith("T"))
    .OrderBy(p => p.Age);

foreach (var person in selectedPeople)
    Console.WriteLine($"{person.Name} {person.Age}");
```

```
Tim 15
Tomas 19
Tom 25
```

Where: определяет фильтр выборки

OrderBy: упорядочивает элементы по возрастанию

OrderByDescending: упорядочивает элементы по убыванию

ThenBy: задает дополнительные критерии для упорядочивания элементов возрастанию

ThenByDescending: задает дополнительные критерии для упорядочивания элементов по убыванию

Select: определяет проекцию выбранных значений

Пример проекции Select:

```
var selectedPeople = people
    .Where(p => p.Name.ToUpper().StartsWith("T"))
    .OrderBy(p => p.Age)
    .Select(p => p.Name);
```

Метод Where фильтрует людей в результате остаются только люди с первой буквой T.

Метод OrderBy упорядочивает людей по возрасту.

Метод Select преобразует людей в имена, т.е. преобразует тип Person в string.

First: выбирает первый элемент коллекции

FirstOrDefault: выбирает первый элемент коллекции или возвращает значение по умолчанию

Single: выбирает единственный элемент коллекции, если коллекция содержит больше или меньше одного элемента, то генерируется исключение

SingleOrDefault: выбирает единственный элемент коллекции. Если коллекция пуста, возвращает значение по умолчанию. Если в коллекции больше одного элемента, генерирует исключение

ElementAt: выбирает элемент последовательности по определенному индексу

ElementAtOrDefault: выбирает элемент коллекции по определенному индексу или возвращает значение по умолчанию, если индекс вне допустимого диапазона

Last: выбирает последний элемент коллекции

LastOrDefault: выбирает последний элемент коллекции или возвращает значение по умолчанию

```
var people = new Person[]
{
   new("Tom", 25), new("Bob", 32), new("Sam", 54),
   new("Tim", 15), new("Tomas", 19), new("Bill", 75)
};
// Первый человек, чей возраст меньше 20
var selectedPeople = people
    .First(p => p.Age < 20);

Console.WriteLine(
            $"{selectedPeople.Name} {selectedPeople.Age}");

// Последний человек, чей возраст меньше 20
var selectedPeople2 = people
    .Last(p => p.Age < 20);

Console.WriteLine(
          $"{selectedPeople2.Name} {selectedPeople2.Age}");
```

```
Tim 15
Tomas 19
```

All: определяет, все ли элементы коллекции удовлетворяют определенному условию

Any: определяет, удовлетворяет хотя бы один элемент коллекции определенному условию

```
var people = new Person[]
{
   new("Tom", 25), new("Bob", 32), new("Sam", 54),
   new("Tim", 15),new("Tomas", 19), new("Bill", 75)
};

// Есть ли в коллекции человек, чей возраст меньше 20?
// В коллекции у всех возраст меньше 20?
Console.WriteLine(
    $"{people.Any(p => p.Age < 20)} " +
    $"{people.All(p => p.Age < 20)}");
```

```
True False
```

Take: выбирает определенное количество элементов

Skip: пропускает определенное количество элементов

TakeWhile: возвращает цепочку элементов последовательности, до тех пор, пока условие истинно

SkipWhile: пропускает элементы в последовательности, пока они удовлетворяют заданному условию, и затем возвращает оставшиеся элементы

```
var people = new Person[]
{
   new("Tom", 25), new("Bob", 32), new("Sam", 54),
   new("Tim", 15), new("Tomas", 19), new("Bill", 75)
};
// Пропустим 1 и заберем 3 объекта из коллекции
var selectedPeople = people
    .Skip(1)
    .Take(3);

foreach (var person in selectedPeople)
    Console.WriteLine($"{person.Name} {person.Age}");
```

```
Bob 32
Sam 54
Tim 15
```

Distinct: удаляет дублирующиеся элементы из коллекции

Except: возвращает разность двух коллекцию, то есть те элементы, которые создаются только в одной коллекции

Union: объединяет две однородные коллекции

Intersect: возвращает пересечение двух коллекций, то есть те элементы, которые встречаются в обоих коллекциях

```
var people = new Person[]
{
   new("Tom", 25), new("Bob", 32), new("Sam", 54),
   new("Tim", 15), new("Tomas", 19), new("Bill", 75)
};
var people2 = new Person[]
{
   new("Tom", 11), new("Bob2", 74), new("Sam", 9),
   new("Tim2", 32), new("Tomas2", 44), new("Bill2", 31)
};
```

Люди из коллекции people кроме людей из people2

```
var selectedPeople = people
    .Except(people2);

foreach (var person in selectedPeople)
    Console.WriteLine(
     $"{person.Name} {person.Age}");
```

```
Tom 25
Bob 32
Sam 54
Tim 15
Tomas 19
Bill 75
```

```
var people = new Person[]
{
   new("Tom", 25), new("Bob", 32), new("Sam", 54),
   new("Tim", 15), new("Tomas", 19), new("Bill", 75)
};
var people2 = new Person[]
{
   new("Tom", 11), new("Bob2", 74), new("Sam", 9),
   new("Tim2", 32), new("Tomas2", 44), new("Bill2", 31)
};
// Люди, имена которых есть в коллекции people и в people2
var selectedPeople = people
    .IntersectBy(people2.Select(v => v.Name), p => p.Name);


foreach (var person in selectedPeople)
    Console.WriteLine($"{person.Name} {person.Age}");
```

```
Tom 25
Sam 54
```

Темы для самостоятельного изучения

Рефлексия: https://metanit.com/sharp/tutorial/14.1.php
Управление памятью: https://metanit.com/sharp/tutorial/8.1.php
Клиент-серверные приложения: https://metanit.com/sharp/net/3.4.php
Web приложения: https://metanit.com/sharp/aspnet6/1.1.php

Бесплатные курсы по программированию

https://ulearn.me/

ВСЁ !